Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Davranış modellemesine giriş
  - Şimdiye kadar problem anlaşılıp çözüm tasarımı modellendi; OCL değişmezleri sistemin hep doğru olan özelliklerini tanımladı. Ancak geliştirilen sistemler genellikle tepkiseldir (reaktif): dış uyarılara tepki verirler. Soru şudur: sistem dış uyarılara nasıl tepki vermelidir? Bunu kesin tanımlamak için durum diyagramı kullanılır.
  - Slayt destesi BLM5126-07'dir (69 slayt) (ekrandan, 00:10:00).
- Durum (state) (ekrandan, 00:10:00)
  - Slayttaki tanım: "Verilen bir anda sistem değerleri kümesinin soyut bir tanımı." Kısaca herhangi bir andaki sistemin hâlidir; bütün değişkenlerin aldığı değerlerin karşılığıdır.
  - Örnek: "Dışarıda yağmur yağıyor" ifadesi, zamanda seçilen bir noktada havadaki yağış miktarı için bir soyutlamadır. Hava için iki durum vardır: yağmurlu ya da değil.
- Durum patlaması (state explosion)
  - 10 sokak ve 8 caddeli ızgara biçimli bir kasabada tek araba 80 ayrı yerde bulunabilir. İki arabada durum sayısı kabaca $80 \times 80 = 6400$ olur.
  - 3x3'lük SOS tahtasında her göz S, O ya da boş olabilir; kurallar düşünülmeden $3^9 = 19683$ olası durum vardır.
  - Değişken sayısı ve değer aralığı arttıkça durum uzayı üstel büyür. Her durumu tek tek yazmak mümkün değildir; yalnız olabilecek durumlar modellenir.
- Olay (event)
  - Durum değişikliğine yol açan tekil, ani, atomik ve fark edilebilir oluşumdur. Dış ya da iç uyaran olabilir.
  - Asenkron olaylar rastgele, herhangi bir anda oluşur, yığılabilir ya da zamana yayılabilir. Senkron olaylar bir sayaca ya da saate bağlıdır, periyodiktir (ör. her saat başı alarm).
  - Olayın temeli sistemde durum değişikliği yaratmasıdır; buna durum geçişi (state transition) denir. Örnek: sınav sonucunu görünce "liseden mezun" durumundan "üniversite adayı" durumuna geçmek.
  - Kullanıcı (butona basmak, sürüklemek, yazmak), veri değişikliği (termostatta sıcaklığın 100 dereceyi geçmesi) ve zaman olay yaratabilir. Olaylar biriktirilebilir; hepsine anında tepki vermek gerekmez.
  - UML'deki dört olay tipi: sinyaller (asenkron bildirimler), metot çağrıları (senkron işlem başlatma), veri durumları (data condition) ve zaman akışı.
- Durum mu olay mı? (ekrandan, 00:20:00)
  - Jesse Owens'ın Berlin'de 1936'da 100 metre olimpiyat şampiyonu olması: durum. Rekoru kırması ise bir olaydır.
  - Mor renk: hiçbiri, yalnız bir özelliğin değeri.
  - Binanın yangın söndürme sisteminin yangın nedeniyle devreye girmesi: olay (sistem bekleme durumundan söndürme durumuna geçer).
  - Telefonun çalması: olay. Yağmurlu bir gün: durum. Saatin zilinin 12'yi vurması: olay.
  - Uluslararası Uzay İstasyonu: hiçbiri, yalnız çok karmaşık bir nesne.
  - Bilgisayarın ekran koruyucusunun açılması: hoca bunu tetikleyici bir olay olarak açıklar.
- Reaktif sistemler ve üç seviye
  - Olaylara tepki veren sistemlere reaktif sistemler denir. Normalde işlerin sırasını sis-

tem kendisi kontrol eder; reaktif sistemlerde uyaranlar dışarıdan gelir.
  - Kombinatoryal sistemler: sonraki durum yalnız girdilerle belirlenir; geçmiş olaylar etkilemez. Karar tablosu ya da karar ağacıyla modellenir; ikisi aynı bilgiyi verir.
  - Ardışık (sequential) sistemler: bellek ya da tarihçe vardır; aynı olay mevcut duruma göre farklı sonuç verir. Sonlu durum makinesiyle modellenir.
  - Eş zamanlı (concurrent) sistemler: çok sayıda olay ve geçiş herhangi bir anda oluşabilir; periyodik bir yapı yoktur. Genelde tasarlanan sistemler bunlardır.
- Karar tablosu: atölye örneği (ekrandan, 00:22:40)
  - Üç girdi (ana şalter, ışık şalteri, güç şalteri) ve iki çıktı (ışık, motor). Ana şalter kapalıysa hiçbir şey çalışmaz; açıksa ışık şalteri ışığı, güç şalteri motoru açar.
- Garaj kapısı: ardışık sistem (ekrandan, 00:27:20)
  - Bir motor kapıyı yukarı iter, aşağı çeker ya da durur; tek bir düğme, "kapı açıldı" ve "kapı kapandı" sensörleri vardır. Düğmeye basıldığında olacak şey mevcut duruma bağlıdır: kapı yukarı çıkarken durur, aşağı inerken durur ("altından kedi geçiyor"), dururken ters yönde hareket eder.
  - Durum geçiş tablosu, durum ve olay arttıkça okunmaz hâle gelir; bunun yerine durum geçiş diyagramı çizilir. Düğümlerin konumunun anlamı yoktur.
  - Olay olmadan güvenlik gibi nedenlerle yapılan otomatik geçişlere epsilon geçiş denir.
- Telefon örneği (ekrandan, 00:30:00)
  - Durumlar: telefon kapalı (ON HOOK, çift çizgili, başlangıç), telefon açık (OFF HOOK), tuşlanıyor (DIALING, kendine geçişli), çalıyor (RINGING), meşgul (BUSY), bağlı (CONNECTED). Diyagramda gereksiz etiketli geçişler vardır.
- Durum geçiş diyagramlarının sorunları
  - n durum ve m olay için $n \times m$ mertebesinde olası geçiş vardır; durum sayısı üstel artar; iç içe yapı ve sayma gibi kontroller gösterilemez.
- Durum diyagramları (statecharts)
  - David Harel tarafından sonlu durum makinesini iyileştirmek ve karmaşıklığı yönetmek için geliştirildi. UML'de vardır, CASE araçları destekler.
  - Durumlar kenarları yuvarlatılmış dörtgenlerle, son durum içi dolu noktanın çevresine bir çerçeve çekilerek gösterilir.
  - Eklenen özellikler: derinlik (iç içe gruplama), eş zamanlılık, koşullu geçişler, giriş-çıkış eylemleri ve aktiviteler, olay parametreleri, geçmişi hatırlama (history) ve varsayılan durum.
- İç içe gruplama (ekrandan, 00:35:00 ve 00:40:00)
  - A, B, C örneğinde A ve C, D adlı bir üst durumda gruplanır. A'dan ve C'den B'ye giden iki `f` oku yerine D'den tek bir `f` oku çıkar; D'nin varsayılanı A'dır.
  - Isıtma-soğutma örneğinde Activating ve Active durumları Heating grubunda toplanır; gruptan tek ok çıkarılarak ok sayısı azaltılır.
  - Printer örneği: Idle, Maintenance ve Active üst durumları; Active içinde Validating (varsayılan), Selecting, Processing ve Printing. `[continue]` ve `[not continue]` koşullu geçişleri nesnenin Boolean özelliği üzerinden test edilir. `entry / readCard` ve `exit / ejectCard` eylemleri: Active'e nasıl girilirse girilsin önce kart okunur, nasıl çıkılırsa çıkılsın kart çıkarılır.
- Eş zamanlılık (ekrandan, 00:41:30)
  - Eş zamanlı çalışan parçalar kesikli çizgiyle ayrılır. Y durumunun A (B, C) ve D (E, F, G) adlı iki eş zamanlı parçası vardır. Düzleştirilmiş eşdeğerde B,E, B,F, B,G, C,E, C,F, C,G gibi çarpım durumları ve çok sayıda ok oluşur.
  - Eş zamanlılık sayesinde karmaşıklık $M \times N$ yerine $M + N$ mertebesine iner.
  - UML örneği: Maintenance durumu içinde Testing ve Commanding alt makineleri. İki parça da son durumuna ulaşınca Idle'a dönülür.
- Senkronizasyon: yayım (ardarda) olaylar (ekrandan, 00:46:00)
  - A, D ve H üç eş zamanlı makine. A'daki `f/g` geçişi: `f` olayı olduğunda `g` olayı da tetiklenir; D makinesi E'deyse G'ye geçer. H'de `m/e` ve `n/f` geçişleri vardır.
  - Olaylar genel görünürdür; bütün eş zamanlı makineler bunlardan haberdardır. Buna yayım ya da ardarda olaylar (broadcast cascade) denir.
- Koşullu geçişler ve özel geçişler (ekrandan, 00:50:00)
  - Koşullar köşeli parantezle yazılır, genelde Boolean'dır, sürekli izlenir ve doğru olduğu anda geçiş olur. `in` ve `not in` ile başka bir eş zamanlı makinenin durumu sınanabilir. Zaman koşulu da verilebilir (ör. her gün belirli bir saatte self-test).
  - Idle durumunda `after: 2 seconds / send c.isAlive` ile kendine geçiş: iki saniyede

bir “yaşıyorum” sinyali.
  - `targetAt(p)[isThreat] / t.addTarget(p)`: geçişe neden olan olayın `p` parametresi, tetiklenen metoda aktarılır.
- Geçmiş (history)
  - Command ve Backup durumları arasında geçişte Backup içinde en son kalınan alt durum (Collecting, Copying, CleaningUp) `H` ile hatırlanır. `H*` iç içe bütün seviyelerde geçmişi hatırlar; güçlüdür ama okunurluğu ve takibi zorlaştırır.
- UML durum tanımı (ekrandan, 00:53:00)
  - En üstte durumun adı; `entry` ve `exit` eylemleri; `newTarget` gibi girdi-çıktı olmadan kendine geçiş; `do` ile zaman alan aktiviteler; `defer` ile kuyruğa eklenip sonra işletilen eylemler. Örnek: `Tracking` durumunda `entry / setMode(onTrack)`, `exit / setMode(offTrack)`, `newTarget / tracker.Acquire()`, `do / followTarget`, `selfTest / defer`.
  - Geçiş tanımında kaynak durum, hedef durum, tetikleyen olay, koşul (OCL ile), eylem ve ayrılma/birleşme yer alır.
- Sınıf diyagramı ile ilişki
  - Teorik olarak her sınıfa bir durum diyagramı çizilebilir, ama bu durum patlaması yaratır. Yalnız dış uyaranlara karmaşık tepki veren ya da belirli durumları garanti etmesi gereken sınıflar için çizilir.
- Casio F-91W dijital saat örneği (ekrandan, 01:00:00)
  - Harel’in modeli. En dışta dead ve alive durumları (pil takılınca `bt-in`, çıkarılınca `bt-rm`). Alive içinde main, power (OK, weak), light (on, off), alarm-st (enab, disab), chime-st eş zamanlı makineleri vardır.
  - Displays içinde time, date, alarm, chime, stopwatch ve ayar (update) durumları; A, B, C, D düğmeleriyle geçilir. Stopwatch’a dönüşte `H*` ile kalınan yerden devam edilir.
- Radyo saat örneği: davranış modellemesinin adımları
  - Perceptler: zaman göstergesi, AM/PM ve alarm ışıkları, zamanlayıcı düğmeleri, frekans düğmesi ve beyaz çubuk, ses düğmesi, AM/FM şalteri, dört konumlu mod şalteri (kapalı, radyo, müzikli alarm, bipli alarm), snooze butonu ve hoparlör.
  - Zaman göstergesi (ekrandan, 01:30:00): $12 \times 60 = 720$ durum; iki eş zamanlı makineyle $12 + 60 = 72$ durum. Bu bile fazladır; “saat” tek düğüme soyutlanır. Gösterge ClockTime, WakeTime ve SleepTime durumlu bir alt makinedir. İki yönlü oklar aslında iki ayrı geçiştir.
  - Mod şalteri yalnız komşu konuma kaydırılabilir; bu yüzden On, Off, Music ve Alarm arasında yalnız komşu geçişler (12: sağa, 13: sola) vardır (ekrandan, 01:40:00). Band (AM/FM), Station, VolumeDial, StationKnob, HrSet, MinSet, Setting (None, ClockSet, SleepSet, WakeSet) ve Speaker (Silent, Playing, Beeping) eş zamanlı alt makinelerdir.
  - En dışta plugged ve unplugged durumları vardır; bütün makineler plugged içinde çalışır.
  - Olaylar numaralandırılır (ör. 1 ses ayarını çevir, 2 AM bandını seç, 3 FM bandını seç, 4 istasyon seç, 12 ve 13 mod düğmesini sağa/sola kaydır) (ekrandan, 01:38:30).
  - İç durumlar: snooze sayacı (10 dakika), alarm sayacı (60 dakika sonra alarmı kapatır), uyku sayacı ve her dakika ilerleyen saat sayacı.
  - Diğer iç olaylar (ekrandan, 01:43:50): 19 alarm sayacı bitti → hoparlör sessiz değilse sessize; 20 zaman alarm saatine geldi → Mod:Müzik ise hoparlör çalıyor, Mod:Alarm ise bipliyor; 21 uyuklama sayacı bitti → aynı iki koşul; 22 zaman uyanma saati +1 saate geldi → hoparlör çalıyor ya da bipliyorsa sessize.
  - 20 numaralı olayın geçişinde `[in Music]` ve `[in Alarm]` gibi guard’lar kullanılır; bunlar OCL ile de yazılabilir. Snooze sayacı bitince bir iç olay yayımlanır ve hoparlör bipler (broadcast cascade).
- Geçerleme (ekrandan, 01:50:00)
  - Bir ekiple, tercihen tasarlayanlardan bağımsız, bütün kullanım senaryoları üzerinden gözden geçirme; model kontrolü (diyagramı gerçekleyip testler koşturma); simülasyon. Yeni sorular oluşursa kullanıcıya dönülür.
  - Adımlar: kullanım senaryoları, perceptler, durumlar, harici kontroller ve uyaranlar, uyaran-tepki tablosu, zamanlayıcı gibi iç durumlar, koordinasyon mekanizmaları, eylem ve aktiviteler, geçerleme.
- Yazılım mimarisi nedir? (yeni slayt destesi BLM5126-08, 47 slayt)
  - Antik tapınak, gökdelen ve şato: üçü de barınma sağlar ama farklı malzeme ve işlevsel olmayan gereksinimlere (eldeki malzeme, küçük arsada çok kat, güvenlik) göre farklı

mimarilere sahiptir.
  - Yazılım mimarisi, bir tasarım probleminin en üst, soyut seviyedeki ifadesidir; pratikte kutular ve oklarla temel bileşenler ve bağlantıları gösterilir. Evrensel kabul görmüş bir anlamsal uzlaşı yoktur.
  - Gayri resmî tanım: sistemin alt sistemler ya da modüller hâlinde bölümlenmesi mimari tasarımdır; genellikle katmanlı iyileştirmelerle ve bilinen mimari stiller kullanılarak yapılır.
  - Analiz ve tasarım birbirinden bağımsız tutulmalıdır: analiz modeli belirli bir mimariye ya da donanıma (ör. mainframe) göre kurulmamalı; koleksiyon ve işlem sınıfları tasarımda eklenmelidir.
- Resmî tanımlar (ekrandan, 02:00:00)
  - USP tanımı: yazılım sisteminin organizasyonu hakkındaki önemli kararlar kümesi; yapısal elemanların ve arayüzlerinin seçimi; bu elemanlar arasındaki işbirliklerinde belirtilen davranış; bunların giderek büyüyen alt sistemlere birleştirilmesi (composition) ve bu organizasyona rehberlik eden mimari stil. Mimari, yalnız yapı ve davranışla değil kullanım, işlevsellik, performans, dayanıklılık, yeniden kullanım, anlaşılabilirlik, ekonomik ve teknolojik kısıtlar, ödünleşimler ve estetik kaygılarla da ilgilidir.
  - Perry ve Wolf: gerekçe (rationale) mimarinin parçasıdır. IEEE: sistemin bileşenlerinde, bunların birbirleriyle ve çevreyle ilişkilerinde somutlaşan organizasyonu ve tasarımı yöneten ilkeler.
  - Garlan ve Shaw: mimari = bileşenler (components) + bağlayıcılar (connectors) + düzenleşim (configuration).
- Bileşenler, API'ler, bağlayıcılar
  - Bileşen seçimi: beklenen işlevselliği karşılamalı, mümkünse yeniden kullanılmalı, fiziksel gereksinimlere (ör. paralelleştirme) uymalı; ekibin deneyimi belirleyicidir (Conway yasası). Sistemin değişim yönü de dikkate alınır.
  - API, bileşenin sunduğu ve beklediği arayüzleri tanımlar; genellikle bir programlama diliyle (language binding) ya da OCL, ADL gibi bildirimsel dillerle yazılır.
  - Bağlayıcılar protokol kararları gerektirir: sıra, iletim yönü, yayın mı hedefli mi, hata durumunda geri alma. En basiti senkron, asimetrik prosedür çağrısıdır.
  - Kavramsal mimari: süreç başında, gereksinimler tam anlaşılmadan kurulan yüksek seviyeli mimari.
  - Mimari sapma (architectural drift): geliştirme sırasında planlanandan uzaklaşma. Mimari erozyon (architectural erosion): bakım sürecinde planlı olmayan, belgelenmeyen değişikliklerle bozulma; daha kötüsüdür.
  - Mimari görünümler (ekrandan, 02:10:00): mimari tanım tek bir diyagram değil, bir kararlar kümesidir; sınıf, bileşen, barındırma, nesne, paket, kullanım senaryosu, ardışıl, iletişim ve etkileşim diyagramları katkı sağlar.
- Mimari stiller
  - İsimlendirilmiş karar koleksiyonlarıdır; bileşenlerin ve bağlantılarının nasıl olacağına dair önceki deneyimden gelen tavsiyelerdir.
  - Client-server tanımı (ekrandan, 02:11:50) ve stil kullanmanın faydası: deneyimi kodlamak, geliştirme yükünü azaltmak.
  - Kataloğun kısa tanımları (ekrandan, 02:20:00 ve 02:25:00): abstract data type, batch sequential, blackboard, big ball of mud, client-server, component-based, coroutines, data-centric, domain-driven, implicit invocation (publish-subscribe), layered, master control, message bus, mobile code, object oriented, peer-to-peer, plug-in, pipe and filter, process control, production systems, REST, SOA, shared nothing (sharded), state-transition systems, shared memory, table-driven interpreter. Bunların bazılarından miras alan mikroservisler ve serverless mimariler günümüzün modern çözümleridir.
  - Heterojen sistemler birden fazla stili birlikte kullanır. Aynı uçağın farklı görev versiyonları gibi, farkları az benzerlikleri çok olan sistemlerde ortak kısım bir mimariyle, özelleşen kısımlar ek mimariyle tanımlanır.
  - Mimari tanımlama dilleri (ADL) ve mimari değerlendirme yöntemleri: Architectural Review Board, SAAM, ATAM.

Hocanın Özellikle Vurguladığı Kısımlar

- Durum patlaması

  - Değişkenler arttıkça durum uzayı üstel büyür; her durumu tek tek modellemek mümkün değildir. Statecharts'ın iç içe gruplama ve eş zamanlılık mekanizmaları bu karmaşıklığı yönetmek içindir.
- Düğümlerin konumunun anlamı yoktur
  - Durum diyagramını döndürseniz de aynıdır; "en üstte duruyor" gibi bir anlam yoktur.
- İki yönlü ok aslında iki ayrı geçiştir
  - Radyo saatte ClockTime ile SleepTime arasındaki çift yönlü ok, düğmeye basma ve düğmeyi bırakma olaylarıyla iki ayrı geçişi temsil eder.
- Her sınıfa durum diyagramı çizilmez
  - Yalnız karmaşık, kilitlenme riski olan ve dış uyaranlara özel tepki vermesi gereken yerlerde kullanılır.
- Durum diyagramı kararları değişmez biçimde ifade eder
  - Hangi sınıfın hangi olaya ne tepki vereceği, neleri dinleyeceği ve neleri tetikleyeceği, kim okursa okusun aynı anlaşılır. Snooze'a alarm çalmadan basılınca ne olacağı gibi kararlar da diyagramda söylenmelidir.
- Analiz ve tasarımı ayrı tutun
  - Belirli bir mimariye göre analiz yapılmaz; analiz modeli problemi ve alanı tanımlar.
- Mimari erozyon sapmadan kötüdür
  - Bakım ekibinin günü kurtarmak için yaptığı belgesiz değişiklikler sonraki değişimde yıkımı hızlandırır.
- Big Ball of Mud bir stil değil, stil eksikliğidir
  - Mimari tasarım yapılmazsa ortaya karman çorman bir uygulama çıkar.

Kısa Tekrar Notları

- Durum: verilen bir andaki sistem değerleri kümesinin soyut tanımı. Olay: durum geçişine yol açan ani, atomik oluşum.
- Asenkron olay rastgele; senkron olay periyodik. UML olay tipleri: sinyal, çağrı, veri koşulu, zaman.
- Reaktif sistem seviyeleri: kombinatoryal (karar tablosu/ağacı), ardışık (sonlu durum makinesi), eş zamanlı (durum diyagramı).
- Epsilon geçiş: olay olmadan otomatik geçiş.
- Statecharts (Harel): derinlik, eş zamanlılık, koşullu geçiş, entry/exit/do, olay parametresi, history, varsayılan durum.
- Eş zamanlı parçalar kesikli çizgiyle ayrılır; karmaşıklık $M \times N$ yerine $M + N$.
- Broadcast cascade: `f/g` gibi bir geçiş başka bir olayı tetikler; olaylar bütün makinelere görünür.
- Koşul köşeli parantezde: `[continue]`, `[in Alarm]`.
- `H`: son alt durumu hatırla; `H*`: bütün seviyelerde hatırla.
- UML durum: ad, `entry`, `exit`, kendine geçiş, `do`, `defer`.
- Radyo saat zaman göstergesi: $12 \times 60 = 720$ durum ya da $12 + 60 = 72$; tek düğüme soyutlanır.
- Mimari = bileşenler + bağlayıcılar + düzenleşim (Garlan ve Shaw).
- Drift geliştirmede, erosion bakımda olur.
- Client-server: servis veren ve alan fiziksel olarak ayrı ve bağımsız ölçeklenir; servis alan yalnız servis verene bağımlıdır.
- Heterojen sistem: birden fazla mimari stilin birlikte kullanılması.

Detaylı Açıklamalar

- Durum ve olay arasındaki fark
  - Hoca, durumu tanımlarken kendi adıyla tanımlamaya yaklaştığını (circularity) fark ettiğini söyler ve en sade hâliyle "herhangi bir andaki sistemin hâli" der. Jesse Owens örneğinde koşu sırasında sıradan bir atlet olan sporcu, bitiş çizgisini geçtikten sonra "olimpiyat şampiyonu" durumuna geçer; rekoru kırması olaydır. Yangın söndürme sistemi sensör yüzünden devreye girer; bu bir olaydır ve sistemi bekleme durumundan söndürme durumuna geçirir. Mor renk ise yalnız bir özelliğin değeri, yani bir durumun parçasıdır.

- Karar tablosu (ekrandan, 00:22:40)

Ana şalter	Işık şalteri	Güç şalteri	Işık	Motor
+	+	+	+	+
+	+	-	+	-
+	-	+	-	+
+	-	-	-	-
-	+	+	-	-
-	+	-	-	-
-	-	+	-	-
-	-	-	-	-

  - Aynı bilgi karar ağacıyla da gösterilir: önce ana şalter sorulur (hayırsa her şey kapalı), sonra ışık, sonra güç şalteri.

- Garaj kapısı durum geçiş tablosu (ekrandan, 00:27:20)

Mevcut durum	Girdi	Eylem	Sonraki durum
Kapı kapalı motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor
Motor yukarı çalışıyor	Kapı açıldı sensörü	Motor durdur	Kapı açık motor duruyor
Motor yukarı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor
Kapı açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor aşağı çalışıyor
Motor aşağı çalışıyor	Kapı kapandı sensörü	Motor durdur	Kapı kapalı motor duruyor
Motor aşağı çalışıyor	Düğmeye basıldı	Motor durdur	Kapı yarım açık motor duruyor
Kapı yarım açık motor duruyor	Düğmeye basıldı	Motor çalıştır	Motor yukarı çalışıyor

  - “Kapı yarım açık motor duruyor” iki kez geçer: biri yukarı, biri aşağı hareketten durdurulmuş hâlidir. Hangisi olduğuna göre düğme kapıyı farklı yöne hareket ettirir; bu, ardışık sistemin geçmişe bağlılığını gösterir.

- Radyo saat diyagramının yapısı (ekrandan, 01:40:00)

```
Unplugged <-> Plugged (5, 6)
Plugged:
  Display    : ClockTime <-> WakeTime, ClockTime <-> SleepTime
  Speaker    : Silent, Playing, Beeping
  Setting    : None <-> ClockSet, None <-> SleepSet (8, 9), None <-> WakeSet (10, 11)
  VolumeDial : Wheel (1, kendine geçiş)
  Station    : Frequency
  BandSwitch : AM <-> FM (2, 3)
  StationKnob: Knob (4, kendine geçiş)
  HrSet      : HrUp <-> HrDown (15, 16)
  MinSet     : MSUp <-> MSDown (17, 18)
  Mode       : On <-> Off <-> Music <-> Alarm (12 sağa, 13 sola)
```

  - Hoca, bütün etkileşim noktalarını (perceptleri) birer eş zamanlı alt makine olarak modeller. AM/FM şalteri ve mod şalteri aynı anda değiştirilebildiği için ayrı eş zamanlı makinelerdir; bu sırada ekran saati göstermeye devam eder.
  - İstasyon düğmesi çevrildiğinde üç tepki gerekir: düğmenin yeni konuma gelmesi, beyaz çubuğun kayması ve alıcının kanalının değişmesi. Bu tepkiler, yayımlanan iç olaylarla koordine edilir.

- Mimari stillerden seçmeler
  - Blackboard: herkesin tebeşir alıp yazdığı ortak bir tahta gibi; işbirliği yapan ajanlar fırsatçı kontrolle ortak veri havuzundan istek ve sonuç alır, bırakır.
  - Layered: her katman kısıtlı görünürlükle yalnız üstündekine hizmet eder; hoca işletim sistemi ve X11 örneğini verir, bunun client-server'daki katmanlarla karıştırılmamasını ister.
  - Pipe and filter: Python'daki veri işleme hattı gibi; veri okunur, ön işlenir, PCA gibi bir dönüşümden, bir sınıflama ya da kümeleme algoritmasından ve değerlendiriciden geçer, sonunda grafik çizilir. Veri tek yönde filtreler arasındaki borulardan akar.
  - Process control: geri besleme döngüsü; arabadaki hız sabitleyici 70'e kadar gaz verir, sonra gazdan çeker.
  - REST: katmanlı, durumsuz (her istek bağımsız ele alınır) ve önbelleklenebilir client-server mimarisi; HTTP uygulamalarının çoğu böyledir.
  - SOA: işlevler gevşek ilişkili, durumsuz, keşfedilebilir ve kontratı tanımlı (ör. WSDL) servislerdedir.
  - Implicit invocation: publish-subscribe; sistem olaylara verilen tepkiler olarak kurulur.
  - Plug-in: Eclipse ve SAP gibi sistemlerin üçüncü parti eklentilerle büyümesi; IDE'lerde de çok kullanılır.
- **Ders sonrası notlar:**
  - Hoca iki arabalı örnekte durum sayısını "6400 - 1 = 6399" diye düzeltti, çünkü iki araba aynı yerde bulunamaz. Aynı yerde bulunma hâllerinin sayısı 1 değil 80'dir; arabalar ayırt edilebilirse büyük olasılıkla $80 \times 79 = 6320$, ayırt edilemezse $\binom{80}{2} = 3160$ durum vardır (python ile doğrulandı). Hoca zaten "kabaca 6400" dedi.
  - Hoca Jesse Owens'ın rekorunun "10 saniye limitinin altına düşmesi" olarak kırıldığını söyledi. Owens'ın 1936 Berlin derecesi büyük olasılıkla 10,3 saniyedir; 100 metrede 10 saniyenin altına ilk inen atlet büyük olasılıkla 1968'de Jim Hines'tır.
  - Hoca UML durum tanımında `do`'yu önce "zaman almayan eylemler" için söyledi. Slayta göre aç/kapa gibi zaman almayan eylemler aksiyonlardır; `do` zaman alan aktiviteler içindir.
  - Hoca 21 numaralı olayın (uyuklama sayacı bitti) slaytta yanlış yazıldığını ve hoparlörün sessize geçmesi gerektiğini söyledi. Ancak aynı derste snooze sayacı bitince alarmın yeniden çalması anlatıldı ve diyagramda sayaç bitince hoparlör bipliyor. Slayttaki satır büyük olasılıkla snooze için doğrudur; hocanın kastettiği büyük olasılıkla uyku (sleep) sayacıdır.
  - Hoca SAAM'ı "Software Architecture Assessment Method" diye açtı; yaygın açılımı büyük olasılıkla "Software Architecture Analysis Method"dur. ATAM, Carnegie Mellon Üniversitesi'ndeki SEI tarafından geliştirilmiştir.
  - Broadcast cascade slaytının metni "A'da C'den B'ye F geçişi var" der, ama diyagramdaki `f/g` oku B'den C'ye gider. Hoca da derste bunu ters yazdığını söyledi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.